CHAPTER 1:

INTRODUCTION

INTRODUCTION:

The traditional methods of managing digital identities are plagued by security vulnerabilities, inefficiencies, and a lack of user control. In an increasingly digital world, there is a pressing need for a robust and secure system for managing digital identities. Blockchain technology holds the promise of addressing these challenges, but significant hurdles remain in its adoption and implementation.

The problem at hand is to design and develop a secure, efficient, and user-centric digital identity system using blockchain technology. This system should overcome the shortcomings of traditional identity management approaches, ensuring privacy, security, and user empowerment.

Python:

Python is a high-level, interpreted, and versatile programming language that emphasizes readability and ease of use. Created by Guido van Rossum and first released in 1991, Python has since become one of the most popular and widely adopted programming languages. Known for its clear and concise syntax, Python promotes a readable and expressive coding style, which contributes to its accessibility for beginners and efficiency for experienced developers.

Visual Studio:

Visual Studio is a comprehensive integrated development environment (IDE) developed by Microsoft that supports a variety of programming languages. It provides a robust set of tools and features for software development, making it a popular choice among developers for creating a wide range of applications, including desktop, web, mobile, cloud, and more. Visual Studio offers an extensive suite of productivity-enhancing features, including code editing, debugging, profiling, and testing tools, all within a unified and user-friendly interface.

ABOUT DEPARTMENT

Our organization offers two master degree courses namely (M.Sc) Master in Science (Computer Science) and Master in Computer Applications (M.C.A). The best facilities are provided by the Department for the education of the students. Highly talented faculties are available for the quality education and a very friendly environment is created. Department is very well-equipped with laboratory, class rooms, library. Also department library is equipped with 2000 volumes, 72 e-journals are accessible through UGC-INFLIBNET, 4 popular journals related to Information Technology, Three Computer laboratories are setup for the good education for students. Backups are maintained well even though there are 90 computers in the department. Students are provided with the facility of Wi-Fi so as to get connected to the internet. Our organization is one of the best organizations in quality education.

CHAPTER 2:
SYSTEM STUDY

EXISTING SYSTEM WITH LIMITATION:

There were several ongoing developments and initiatives related to digital identity using blockchain technology. Self-sovereign identity is a concept where individuals have full control over their digital identities. Blockchain plays a crucial role in SSI by enabling users to store and manage their identity information securely. Projects like Sovrin, uPort, and Microsoft's ION have been exploring SSI solutions. Some governments around the world have been exploring blockchain-based digital identity systems to enhance security and privacy for citizens. For example, Estonia's e-Residency program incorporates blockchain for secure digital identities.

PROPOSED SYSTEM WITH OBJECTIVES:

The proposed system for digital identity using blockchain aims to address the challenges and shortcomings of existing identity management systems. It leverages the decentralized and immutable nature of blockchain technology to provide a secure, privacy-enhancing, and user-centric digital identity solution. Here are the key components and features of the proposed system:

FEASIBILITY STUDY:

The prime focus of the feasibility study is evaluating the practicality of the proposed system keeping in mind a number of factors.The following factors are taken into account before decidingin favor of the new system.

1. Economic Feasibility
2. Technical Feasibility
3. Duration

EconomicFeasibility:

The economic feasibility study evaluate the cost of the application development againstthe ultimate income or benefitsgetsfromthe developed system, There mustbe scopes forprofitafter thesuccessfulCompletion ofthe project.

TechnicalFeasibility:

Thetechnical feasibility study compares the level of technology available in the software development firm and the level of technology required for the development of the product.Here the level of technology consists of programming language,the hardwarere sources,other software tools etc.

DURATION:

DESCRIPTION	FROM	TO
Requirements analysis, understating the basic problem statement	04/06/2023	05/07/2023
Scheduling & planning for the project	07/07/2023	15/07/2023
Data Analysis & Exploration	17/07/2023	29/08/2023
Coding & Testing for conditions	01/08/2023	02/09/2023
Resolving the Errors (if found while testing)	04/09/2023	13/09/2023
Documentation & Submission of the final project report.	15/09/2023	30/09/2023

CHAPTER 3:

SYSTEM ANALYSIS

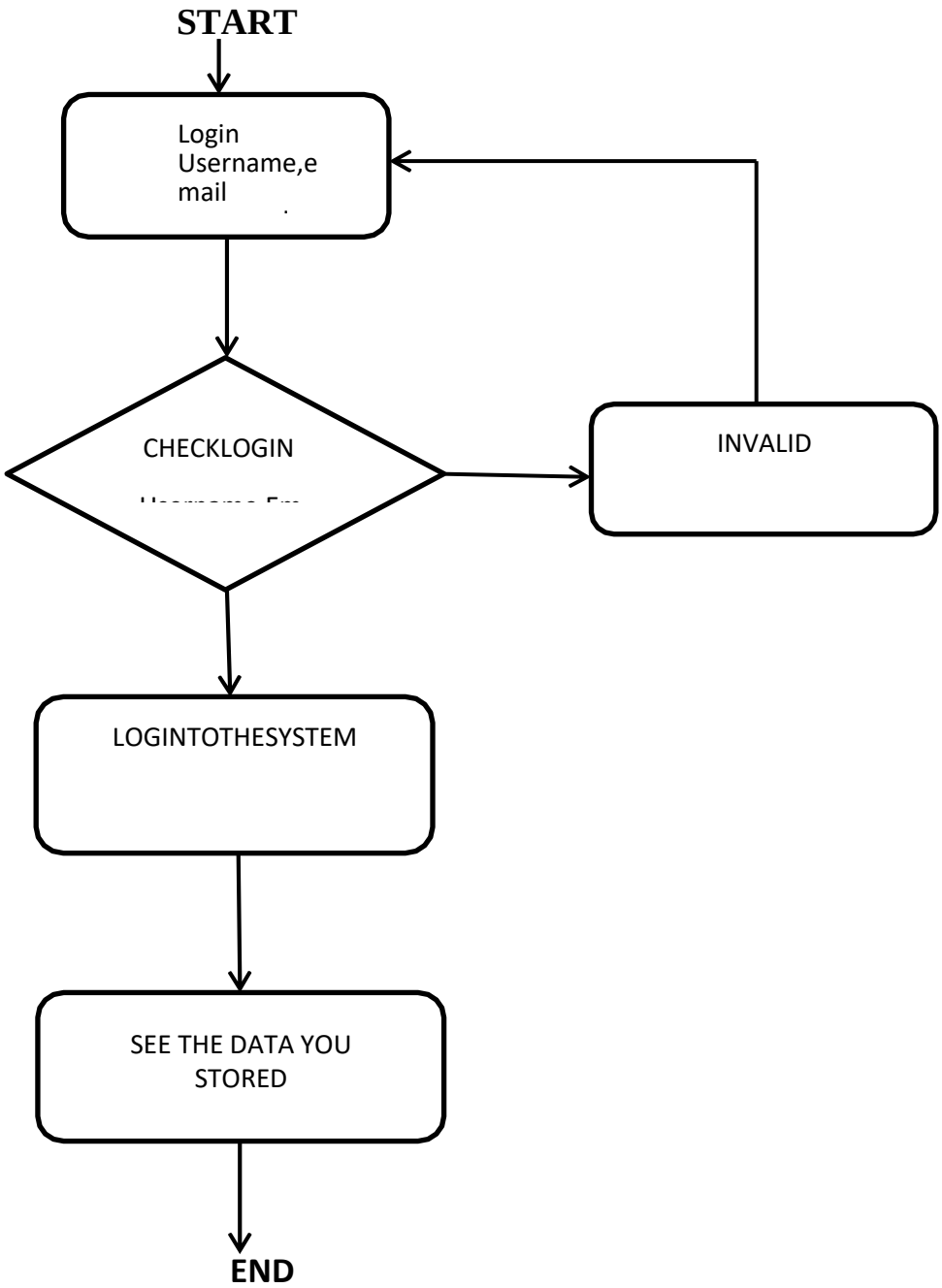


FIG: SYSTEM FLOWCHART

ENTITY RELATIONSHIP DIAGRAM (E-R DIAGRAMS):

A conceptual model describes the essential features of system data. This conceptual model is described modeling method known as Entity Relationship. Entity Relationships is uses three major abstractions to describe data. To describe the basic data-structuring concepts and constraints of the ER model and discuss their use in the design of conceptual schemas for database applications. We also present the diagrammatic notation associated with the ER model, known as "**E-R diagrams**".

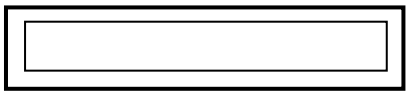
Elements of ER-Diagram

1. **ENTITY:**

An Entity can be any object place, person or class. In ER-Diagram, An Entity is represented using rectangles. Consider an example of an organization. Employee, Manager, Department, Product and many more can be taken as Entities from an organization.

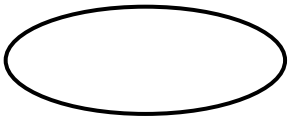
2. **WEAK ENTITY:**

Weak Entity is an entity that depends on another entity. Weak entity doesn't have key attribute of their own. A weak entity, in the context of database design, is an entity that does not have sufficient attributes to form a primary key. It depends on another entity, called a strong entity, to provide part of or all of its primary key



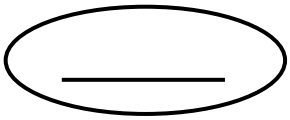
3. **ATTRIBUTE:**

An attribute describes the properties or characteristics of an entity. For Example, name, age, address etc. can be attributes of an Employee. An attribute is represented using ellipse.



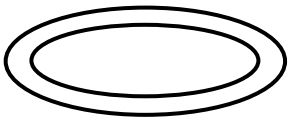
4. KEY ATTRIBUTE:

Key attribute represents the main characteristic of an entity. Is use to represent primary key. Ellipse with underlines represent key attribute.



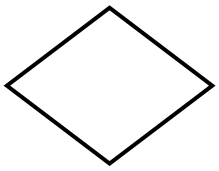
5. MULTIVALUE ATTRIBUTE:

A multi-valued attribute in the context of database design is an attribute that can hold multiple values for a single entity. In other words, a multi-valued attribute can have a set of values associated with a single entity occurrence.



6. RELATIONSHIP:

A Relationship describes the relation between entities. Relationship is represents using diamonds.



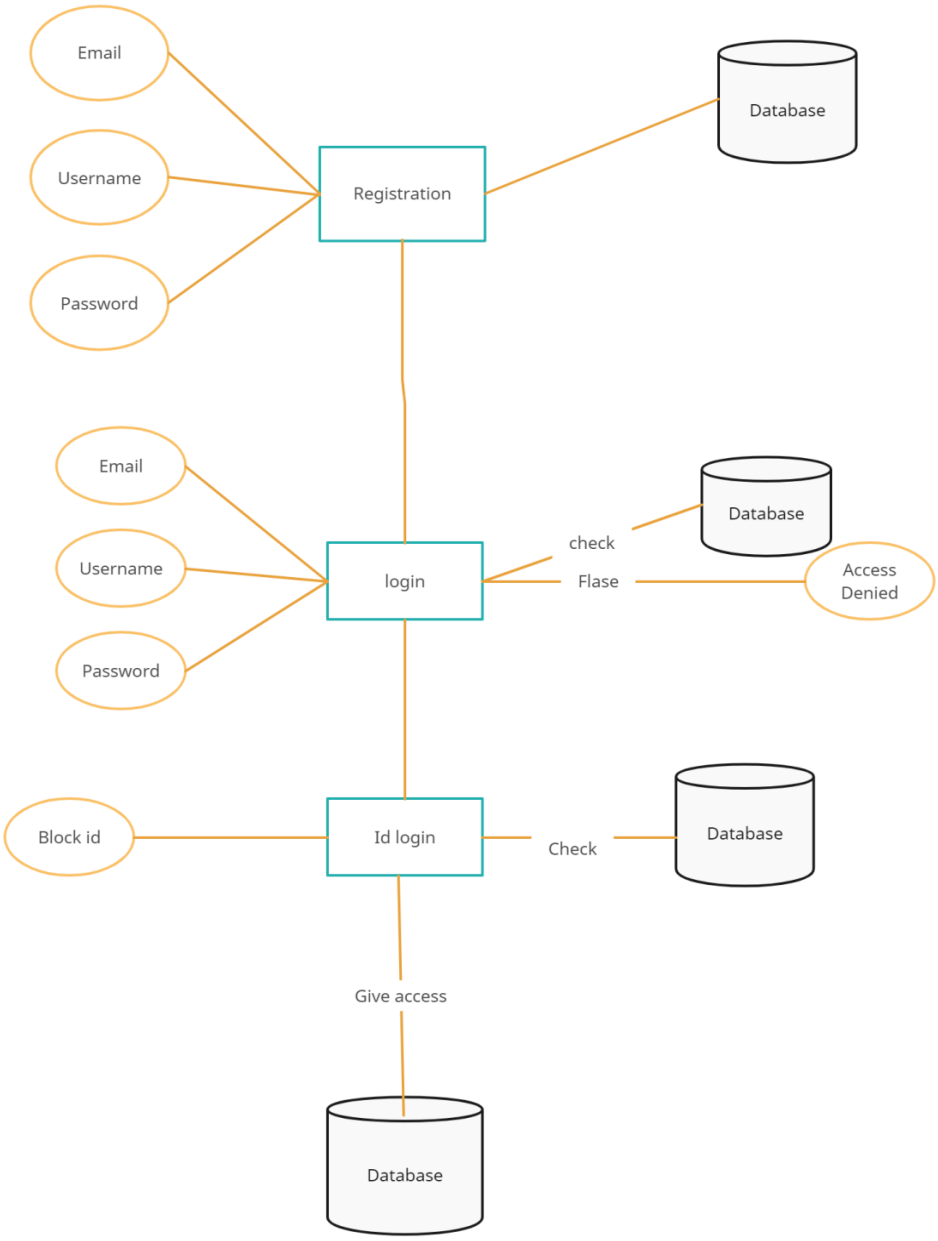


FIG: E-R DIAGRAM

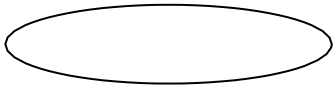
DATA FLOW DIAGRAM (DFDs):

Data flow diagrams are a graphical tool used to describe and analyse the movement of data through a system. DFD's are used to capture the essential feature of both existing real system and future physical implementation of the system.

Different notations used in DFD are:

Functional Processing:

It is represented by an oval. The processing or main transactions are specified by thisnotation.

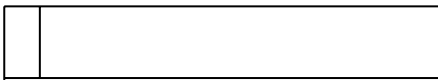


Data Flow:

It is represented by an arrow line and name of the data is specified by the side of the line as label. This is used for data movement.

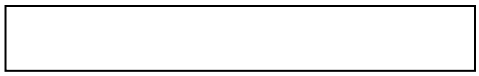
Data Store:

It is represented by one open-end rectangle. The databases used in the system are specified by this notation.



Source or sink:

It is represented by one open-end rectangle. It is used for specifying from where data comes and where it reaches.



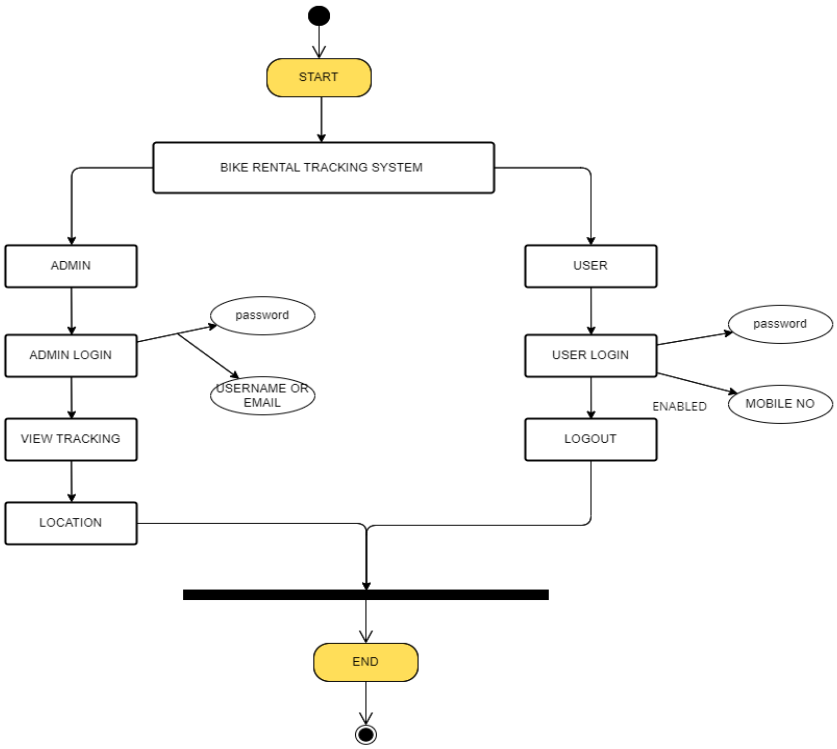


Fig: DFD DIAGRAM

REQUIREMENT SPECIFICATION:

System requirements are the configuration that a system must have in order for a hardware or software application to run smoothly and efficiently. Failure to meet these requirements can result in installation problems or performance problems. The former may prevent a device or application from getting installed, whereas the latter may cause a product to malfunction or perform below expectation or even to hang or crash.

Hardware Requirements:

Hardware basically refers to the item in a PC that can be touched and felt like keyboard, monitor, mouse and the system unit. Additional hardware components that can be added to the PC are modem, printer, scanner etc.

The hardware thus used here are consists of:

Processor	:	intel i3 and above
Hard Disk	:	500GB
RAM	:	4GB & above
Monitor	:	15” LED
Input Devices	:	Keyboard. Mouse.

Software Requirement:

There is use of large software in developing this project. Software is basically the logical program that handles different components which cannot be touched or felt and helps to interact with one another in a Hassle-free manner.

The software used here consists of:

Operating system	:	WINDOWS 10.
Front End	:	HTML.CSS
Middleware	:	FLASK
Back End	:	PYTHON
Platform/Middleware	:	POSTGRESQL

CHAPTER 4:
SYSTEM DESIGN

File/Database Design:

USERS TABLES

COLUMN NAME	COLUMN TYPE	LENGTH	NULL?	CONSTRAINTS
SNO	INT	50	NOT NULL	PRIMARY KEY
NAME	VARCHAR	50	NO	
EMAIL	VARCHAR	50	NO	
PASWORD	VARCHAR	10	NO	

IMAGE TABLES:

COLUMN NAME	COLUMN TYPE	LENGTH	NULL?	CONSTRAINTS
SNO	INT	50	NOT NULL	PRIMARY KEY
TEXT	VARCHAR	50	NO	
IMAGE	VARCHAR		NO	

BLOCK TABLES

COLUMN NAME	COLUMN TYPE	LENGTH	NULL?	CONSTRAINTS
SNO	INT	50	NOT NULL	PRIMARY KEY
BLOCK ID	INT	50	UNIQUE	
HASH VALUE	VARCHAR	50	NOTNULL	
DATE TIME	DATE TIME		NO	

NORMALIZATION

The normalization process takes a relational schema through a series of tests to "certify" whether it satisfies a certain normal form. The process, which proceeds in a top down fashion, by evaluating each relation against the criteria for normal forms and decomposing the relations as necessary can thus be considered as the normalization process or relational design by analysis. The normalization criteria followed in the project include the following:

1. First Normal Form (1NF)
2. Second Normal Form (2NF)
3. Third Normal Form (3NF)

1. First Normal Form:

It is defined to disallow multi-valued attributes, composite attributes and their combinations. It states that the domain of the attribute must include only atomic values and that the value of any attribute in the domain must be a single value from the domain of that attribute. Hence 1NF disallows having a set of values, a tuple of values, a combination of both as an attribute valueof a single tuple. In other words, 1NF disallows

2. Second Normal Form:

The test for 2NF involves testing for functional dependencies whose left-hand attributes are part of the primary-key. If the primary-key has a single attribute then the test need not be applied at all. A relational schema R is said to be in 2NF if every non-prime attribute in R is fully functionally dependent on the primary key of R.**Remedy:** Decompose and setup a new relation for each partial key with its dependent attributes. Make sure to keep a relation with the original primary key and any attributes that are functionally dependent on it.

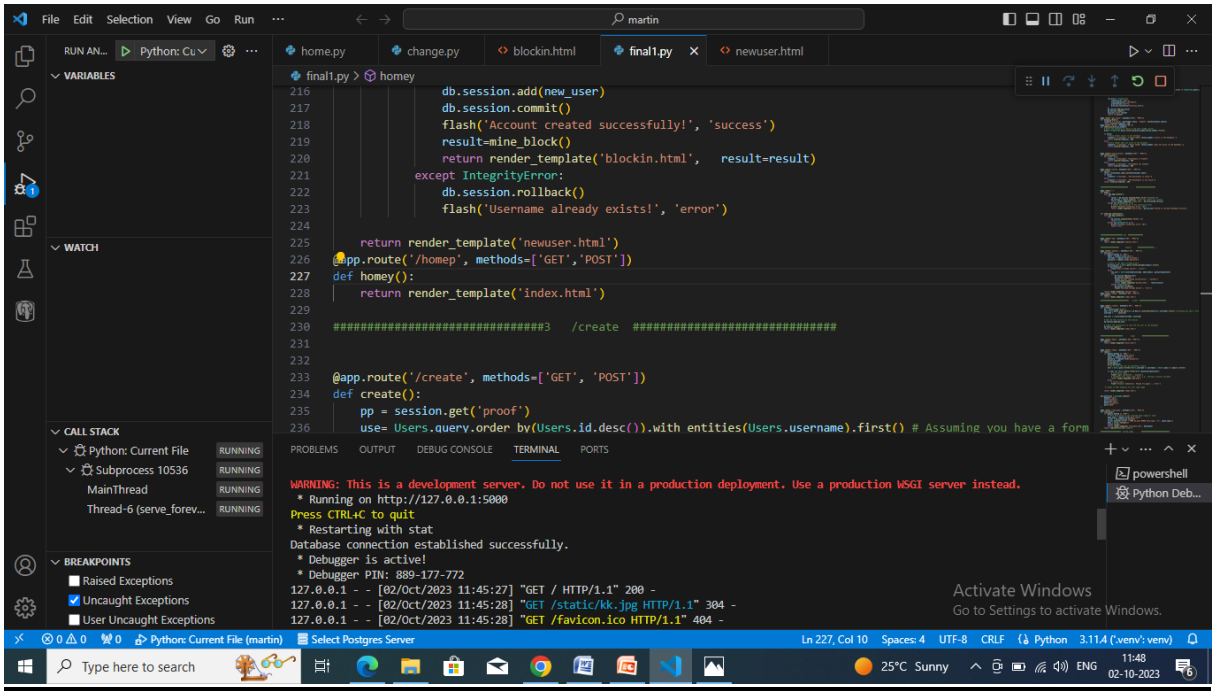
3. Third Normal Form:

This is based on the concept of transitive dependency. A functional dependency X -> Y in a relation R is a transitive dependency if there is a set of attributes Z that is neither a candidate key nor a subset of any key of R, and both X –>Z and Z -> Y hold. In other words, a relation should not have a non-key attribute functionally determined by another non-key attribute (or bya set of non-key attributes).**Remedy:** Decompose and set up a relation that includes the non-key attribute(s) that functionally determine other non-key attributes.

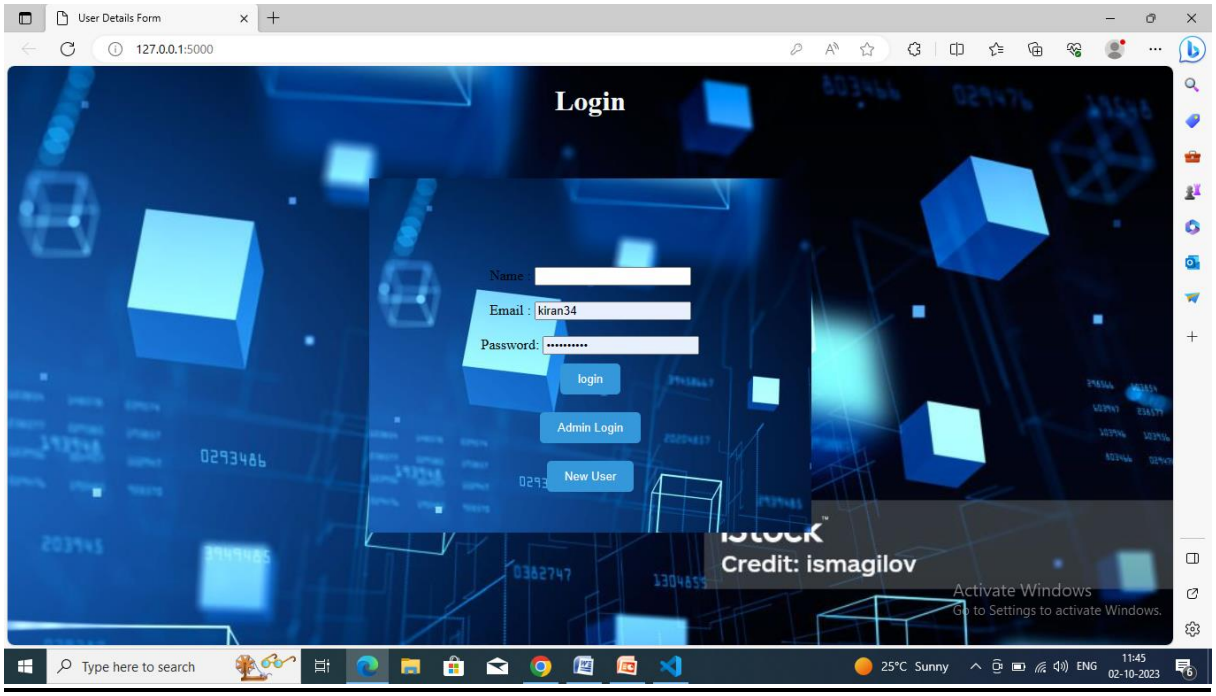
All the relations in our project satisfy fully “functional dependency” property and hence are in the Third Normal Form.

Input / Output form design:

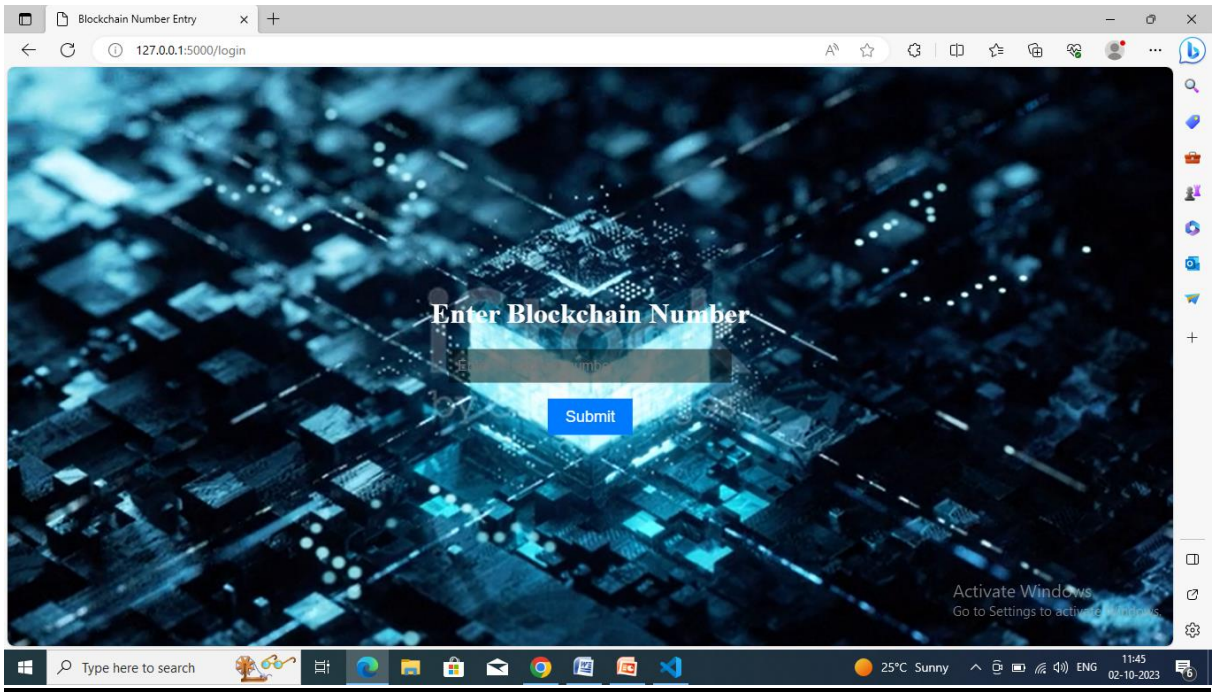
EXECUTED PROGRAM HTTP:



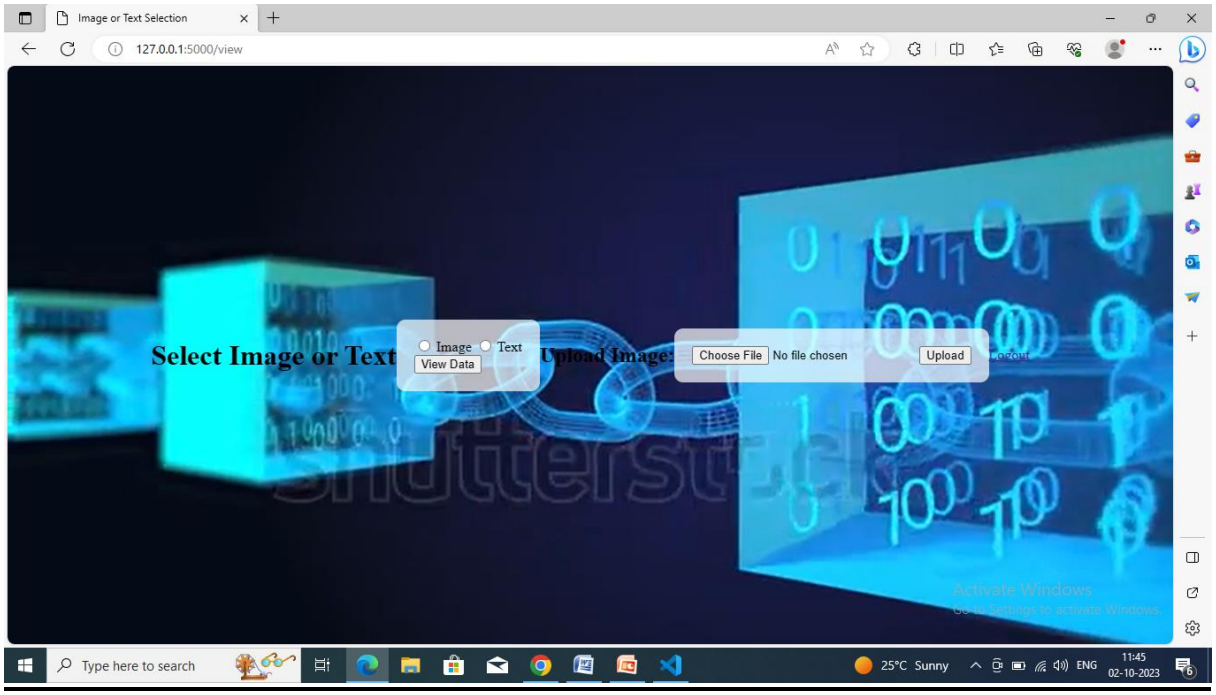
HOME PAGE:



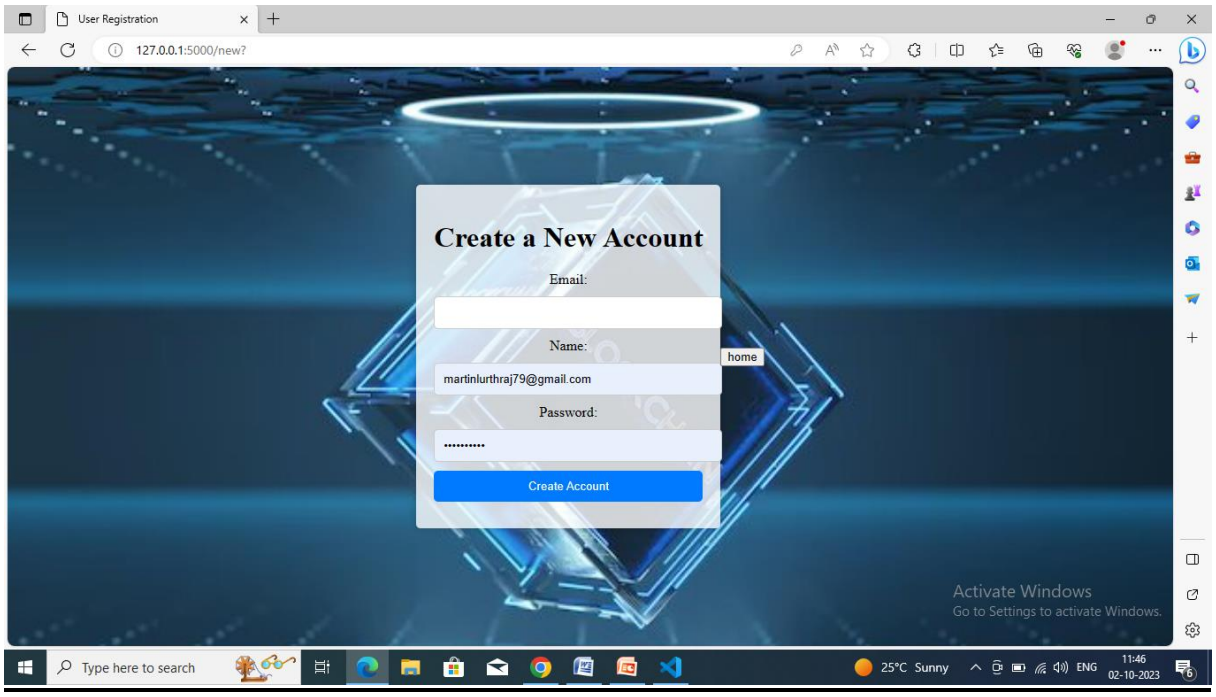
LOGIN PAGE :



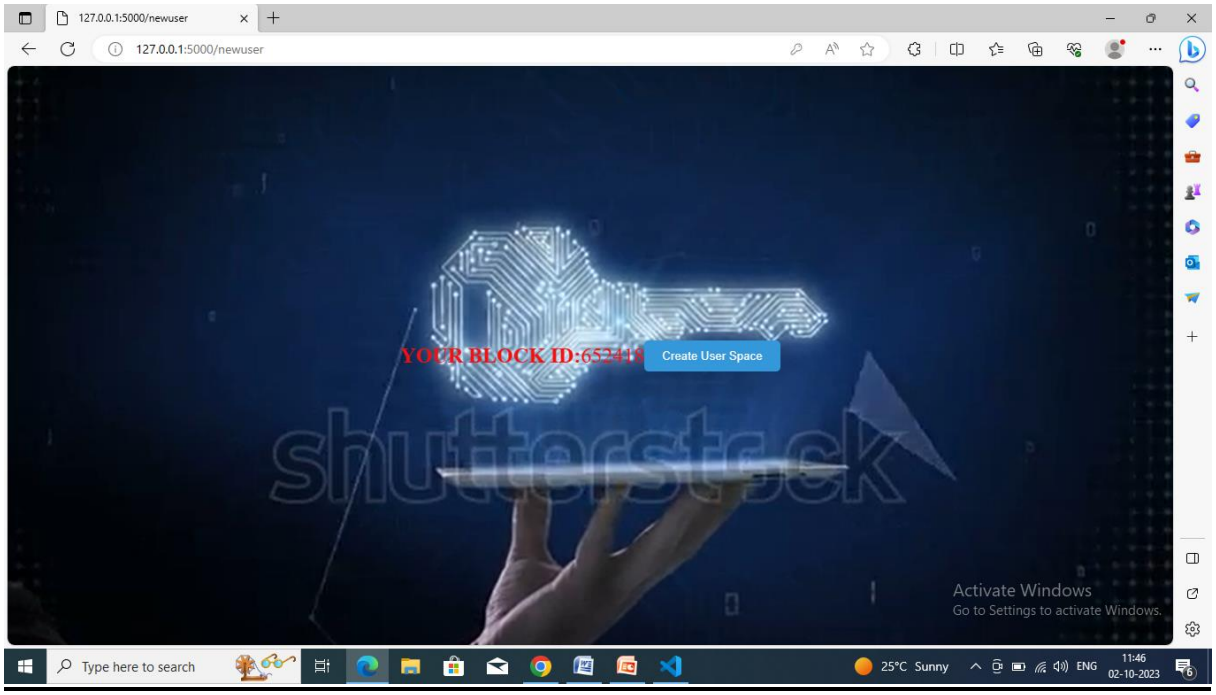
STATUS PAGE:



CREATE NEW USER:



GENTETED BLOCK ID:



CHAPTER 5: CODING

Source/Pseudo Code:

Index.html

```
<!DOCTYPE html>
<html>
<head>
        <title>User Details Form</title>
  <style>
  body {
      background-image: url('static/kk.jpg');
      background-size: cover;
      background-repeat: no-repeat;
      background-attachment: fixed;
  }
  /* Center align content */
  .container {
      text-align: center;
      padding: 100px;
      background-color: aliceblue;
      width: 300px;
```

```
        height: 200px;
        position: absolute;
        top: 50%;
        left: 50%;
        transform: translate(-50%, -50%);
        background-image: url('static/kk.jpg');
    }
    /* Style buttons */
    button {
        background-color: #3498db;
        color: white;
        padding: 10px 20px;
        border: none;
        border-radius: 5px;
        margin: 10px;
        cursor: pointer;
    }
    button:hover {
        background-color: #2980b9;
    }
</style>
```

```
    <script>
        function validatePassword() {
            var password = document.getElementById("password").value;
            var passwordRegex = /^(?=.*[a-zA-Z])(?=.*\d)(?=.*[@$!%*?&])[A-Za-
z\d@$!%*?&]{8,}$/;
            if (password.match(passwordRegex)) {
                return true;
            } else {
                alert("Password must contain at least 8 characters, including one alphabet letter, one
number, and one special character.");
                return false;
            }
        }
    </script>
</head>
<body>
        <h1 style="text-align: center; color: rgb(245, 248, 246); ">Login</h1>
    <div class="container">
        <form id="userDetailsForm" action="/login" method="POST">
            <label for="name">Name :</label>
            <input type="text" id="name" name="name" required><br><br>
            <label for="email">Email  :</label>
```

```
        <input type="email" id="email" name="email" required><br><br>
        <from onsubmit="return validatePassword()">
            <label for="password">Password:</label>
            <input type="password" id="password" name="password" required>
            <button type="submit" onclick="submit">login</button>
        </from>
    </form>
    <form action="/admin_login" method="GET">
        <button type="submit">Admin Login</button>
    </form>
    <form action="/new" method="GET">
        <button type="submit">New User</button>
    </form>
  </div>
 <script>
    const users = [];
    function addUser() {
        const name = document.getElementById("name").value;
        const email = document.getElementById("email").value;
        const password = document.getElementById("password").value;
        const user = {
```

```
            name: name,
            email: email,
            password: password
        };
        users.push(user);
        displayUserList();
        clearForm();
    }
    function displayUserList() {
        const userList = document.getElementById("userList");
        userList.innerHTML = "";
        users.forEach((user, index) => {
            const listItem = document.createElement("li");
            listItem.innerHTML = `User ${index + 1}:<br>Name: ${user.name}<br>Email:
${user.email}<br>Password: ${user.password}<br>`;
            userList.appendChild(listItem);
        });
    }
    function clearForm() {
        document.getElementById("name").value = "";
        document.getElementById("email").value = "";
        document.getElementById("password").value = "";
```

```
        }
    </script>
</body>
</html>
```

Block.html

```
<!DOCTYPE html>
<html>
<head>
    <title>block</title>
</head>
<body style="background-image:url('static/number.jpg');">
    <h1 style="color: blanchedalmond;">Enter a Number</h1>
    <form style="align-items: center;">
        <label style="color: rgb(0, 255, 149);" for="numberInput">Enter your Block
Number:</label>
        <input type="number" id="numberInput" name="number"  step="1" required>
        <br><br>
        <input type="submit" value="Submit">
    </form>
</body>
</html>
```

Blockin.html

```
<!DOCTYPE html>

<html>

<head>

</head>

<body>

    <h1>YOUR BLOCK ID:</h1>

    <p>{{ result }}</p>

    <form action="/newuser" method="POST">

        <!-- Your form elements here -->

        <button type="submit">create user space</button>

    </form>

</body>

</html>
```

ii.html

```
<!DOCTYPE html>

<html>

<head>
```

```
    <title>Image or Text Selection</title>
    <link rel="stylesheet" type="text/css" href="{{ url_for('static', filename='styles.css') }}">
</head>
<body>
    <h1>Select Image or Text</h1>
    <form action="/view_data" method="POST">
        <label>
            <input type="radio" name="data_type" value="image"> Image
        </label>
        <label>
            <input type="radio" name="data_type" value="text"> Text
        </label>
        <br>
        <input type="submit" value="View Data">
    </form>
    <h2>Upload Image:</h2>
    <form action="/upload_image" method="POST" enctype="multipart/form-data">
        <input type="file" name="image">
        <input type="submit" value="Upload">
    </form>
    <a href="/logout">Logout</a>
```

```
</body>

</html>
```

Home.py

```
from flask import Flask, render_template, request, redirect, url_for,
jsonify,flash,send_file,flash

from flask_sqlalchemy import SQLAlchemy

from flask import Flask, session

from flask_session import Session

from sqlalchemy.exc import OperationalError

from sqlalchemy import text

from flask_migrate import Migrate

import datetime

import hashlib

import json

from flask_wtf import FlaskForm  # Add this import

from flask_bcrypt import Bcrypt

from sqlalchemy.exc import IntegrityError

import psycopg2

from io import BytesIO

from sqlalchemy import desc

 app= Flask(__name__)

# Configure the SQLAlchemy database URI
```

```
#app.config['SQLALCHEMY_DATABASE_URI'] =
'postgresql://postgres:1234@localhost/blockchain'

app.config['SQLALCHEMY_DATABASE_URI'] =
'postgresql://postgres:1234@localhost:5433/login'

app.config['SECRET_KEY'] = 'Martin'

#session

app.config['SESSION_TYPE'] = 'filesystem'

Session(app)

db = SQLAlchemy(app)

migrate = Migrate(app, db)

# Define your data model

class Product(db.Model):

    id = db.Column(db.Integer, primary_key=True)

    name = db.Column(db.String(255), nullable=False)

    price = db.Column(db.Float, nullable=False)

class Properties(db.Model):

    pro_id = db.Column(db.Integer, primary_key=True)

    index = db.Column(db.Integer, nullable=False)

    timestamp = db.Column(db.String(50), nullable=False)

    proof = db.Column(db.Integer,unique=True, nullable=False)

    previous_hash = db.Column(db.String(64), nullable=False)

class Users(db.Model):
```

```
    username = db.Column(db.String(80),unique=True, primary_key=True)

    email = db.Column(db.String(128), unique=True, nullable=False)

    password = db.Column(db.String(128), nullable=False)

class up(db.Model):

    number = db.Column(db.Integer, primary_key=True)

    username = db.Column(db.String(255), db.ForeignKey('Users.username'), nullable=False)

    poof= db.Column(db.Integer, db.ForeignKey('Properties.proof'), nullable=False)

#################################          create   ##########################3

@app.route('/create', methods=['GET','POST'])

def create():

    from home import db

    pp= session.get('poof')

    las= Users.query.order_by(Users.username.desc()).first()

  # Get the username from the request (e.g., from a query parameter)

    # Query the 'User' table to get the last added record for the specified username

   # las = Users.query.filter_by(username=username).order_by(Users.username.desc()).first()

    print(las)

    print(pp)

    new_user = up(username=las, poof=pp)

        # Add the User instance to the session

    db.session.add(new_user)
```

```
        # Commit the transaction to save the new user to the database
    db.session.commit()
    return render_template('index.html')
#home page
@app.route('/')
def home():
    with app.app_context():
        try:
            result = db.session.execute(text('SELECT version()'))
            db_version = result.scalar()  # Get the result as a scalar
            return render_template('index.html', db_version=db_version)
        except OperationalError as e:
            # Handle the exception if the connection fails
            print(f"Database connection error: {e}")
            return render_template('hello.html', db_version='Failed to retrieve database version')
def check_db_connection():
    with app.app_context():
        try:
            db.session.execute(text('SELECT 1'))
            return True
        except OperationalError as e:
```

```
        print(f"Database connection error: {e}")
        return False
####################3      newuser      ##################____
@app.route('/newuser', methods=['GET', 'POST'])
def new_user():
    if request.method == 'POST':
        email = request.form['email']
        username = request.form['name']
        password = request.form['password']
        # Check if the email already exists
        existing_user = Users.query.filter_by(email=email).first()
        if existing_user:
            flash('Email already exists!', 'error')
        else:
            new_user = Users(username=username, email=email, password=password)
            try:
                db.session.add(new_user)
                db.session.commit()
                flash('Account created successfully!', 'success')
                result=mine_block()
                return render_template('blockin.html',   result=result)
```

```
        except IntegrityError:
            db.session.rollback()
            flash('Username already exists!', 'error')
    return render_template('newuser.html')
@app.route('/add_product', methods=['GET'])
def add_product_page():
    return render_template('add_product.html')
@app.route('/add_product', methods=['POST'])
def add_product():
    if request.method == 'POST':
        name = request.form.get('name')
        price = float(request.form.get('price'))
        # Create a new product and add it to the database
        new_product = Product(name=name, price=price)
        db.session.add(new_product)
        db.session.commit()
        return redirect(url_for('add_product_page')
## ========================================================== ##
class Blockchain:
    def __init__(self):
        self.chain = []
```

```
        self.create_block(proof=1, previous_hash="0")


    def create_block(self, proof, previous_hash):
        block = {
            "index": len(self.chain) + 1,
            "timestamp": str(datetime.datetime.now()),
            "proof": proof,
            "previous_hash": previous_hash,
        }
        self.chain.append(block)
        return block


    def print_previous_block(self):
        return self.chain[-1]
    def proof_of_work(self, previous_proof):
        new_proof = 1
        check_proof = False
        while check_proof is False:
            hash_operation = hashlib.sha256(
                str(new_proof * 2 - previous_proof * 2).encode()
            ).hexdigest()
```

```
        if hash_operation[:5] == "00000":
            check_proof = True
        else:
            new_proof += 1
    return new_proof
def hash(self, block):
    encoded_block = json.dumps(block, sort_keys=True).encode()
    return hashlib.sha256(encoded_block).hexdigest()
def chain_valid(self, chain):
    previous_block = chain[0]
    block_index = 1
    while block_index < len(chain):
        block = chain[block_index]
        if block["previous_hash"] != self.hash(previous_block):
            return False
        previous_proof = previous_block["proof"]
        proof = block["proof"]
        hash_operation = hashlib.sha256(
            str(proof * 2 - previous_proof * 2).encode()
        ).hexdigest()
        if hash_operation[:5] != "00000":
```

```
            return False
        previous_block = block
        block_index += 1
    return True
blockchain = Blockchain()


############################## mine_block ##############################
@app.route("/mine_block", methods=["GET", "POST"])
def mine_block():
  if request.method == 'POST':
    previous_block = blockchain.print_previous_block()
    previous_proof = Properties.query.order_by(Properties.pro_id.desc()).first().proof if
Properties.query.count() > 0 else 0
    proof = blockchain.proof_of_work(previous_proof)
    previous_hash = blockchain.hash(previous_block)
    block = blockchain.create_block(proof, previous_hash)
    new_block = Properties(
        index=block["index"],
        timestamp=block["timestamp"],
        proof=block["proof"],
        previous_hash=block["previous_hash"],
    )
```

```
        db.session.add(new_block)
        db.session.commit()
        session['proof']=proof
        return str(proof)
@app.route("/get_chain", methods=["GET", "POST"])
def display_chain():
    response = {"chain": blockchain.chain, "length": len(blockchain.chain)}
    return jsonify(response), 200
@app.route("/block", methods=['GET'])
def check_block(block_number):
    # Query the database to check if the block number exists
    block = Properties.query.filter_by(block_number=block_number).first()
    if block:
        # Block number exists in the database
        response = {'message': f'Block number {block_number} exists in the database.'}
        return jsonify(response), 200
    else:
        # Block number does not exist in the database
        response = {'message': f'Block number {block_number} does not exist in the database.'}
        return jsonify(response), 404
@app.route("/new", methods=["GET", "POST"])
```

```
def new():
    return render_template('newuser.html')
@app.route("/search_block", methods=["GET", "POST"])
def search_block():
    if 'proof' == 5:
        response = {"message": "blockchain is found"}
        return jsonify(response), 200
    else:
        response = {"message": "blockchain not found"}
        return jsonify(response), 200
@app.route("/valid", methods=["GET", "POST"])
def valid():
    valid = blockchain.chain_valid(blockchain.chain)
    if valid:
        response = {"message": "The Blockchain is valid."}
    else:
        response = {"message": "The Blockchain is not valid."}
    return jsonify(response), 200
#########################        login        ################################
@app.route('/login', methods=['GET', 'POST'])
def login():
```

```
    if request.method == 'POST':
        username= request.form['name']
        email = request.form['email']
        password = request.form['password']
        print(username)
        print(email)
        print(password)
        # Try to find the user by username or email
        user = Users.query.filter((Users.username == username) | (Users.email == email)).first()
        if user and (Users.query.filter(Users.password==password)):
            # Successful login
            flash('Login successful', 'success')
            # Implement your login logic here, e.g., setting a session variable
            return render_template('add.html')
        else:
            # Failed login
            flash('Invalid credentials. Please try again.', 'error')


     # Create an HTML template for your login page
    return render_template('index.html')
db_connection = psycopg2.connect(
```

```
    dbname="login",
    user="postgres",
    password="1234",
    host="localhost",
    port="5433"
)
######################### view_data          ##############################
@app.route('/view_data', methods=['GET', 'POST'])
def view_data():
    if request.method == 'POST':
        # Get the table name and data type from the form
        table_name = request.form['table_name']
        data_type = request.form['data_type']
        # Construct the SQL query dynamically with the provided table name
        query = f"SELECT * FROM {table_name} WHERE data_type = %s"
        cursor = db_connection.cursor()
        cursor.execute(query, (data_type,))
        data = cursor.fetchall()
        cursor.close()
        return render_template('view_data.html', data=data)
    return redirect(url_for('index'))
```

```
######################  upload_image        ################################
@app.route('/upload_image/<table_name>', methods=['POST'])
def upload_image(table_name):
    if 'image' in request.files:
        image_file = request.files['image']
        if image_file.filename != '':
            # Read the image file and store it in the specified table
            image_data = image_file.read()
            data_type = 'image'
            # Construct the SQL query dynamically with the provided table name
            query = f"INSERT INTO {table_name} (data_type, content) VALUES (%s, %s)"
            cursor = db_connection.cursor()
            cursor.execute(query, (data_type, image_data))
            db_connection.commit()
            cursor.close()
    return redirect(url_for('index'))


#######################        veiw  ################################
@app.route('/view', methods=['GET'])
def view():
```

```
    return render_template('ii.html')
#################### get_image ##########################
@app.route('/get_image/<table_name>/<int:id>')
def get_image(table_name, id):
    cursor = db_connection.cursor()
    cursor.execute(f"SELECT content FROM {table_name} WHERE id = %s", (id,))
    image_data = cursor.fetchone()
    cursor.close()
    if image_data:
        return send_file(BytesIO(image_data[0]), mimetype='image/jpeg')  # Adjust mimetype
based on your image format
    return 'Image not found'
@app.route("/jls_extract_def", methods=["GET", "POST"])
def jls_extract_def():
    response = {"message": "Your custom message here"}  # You can customize the message
    return jsonify(response), 200
if __name__ == '__main__':
    if check_db_connection():
        print("Database connection established successfully.")
    else:
        print("Failed to establish a database connection.")
    app.run(debug=True)
```

CHAPTER 6:
SYSTEM TESTING

Preparation of Test Data & Test:

Test Case	Input	Expected O/P	Actual O/P	Result
1	Valid Username and Password	It should display respective page according to usertype.	Respective Home is displayed .	Passed
2	InvalidUsername and Password	It should give appropriate error message saying "Enter proper User-Idand Password"	Error message Displayed	Passed
3	Add/delete/ Inventory items	It should Add/delete Inventory items	Added/Deleted inventory items successfully	Passed
4	Generate Blockchain ID	It should Add Blockchain ID	Added Blockchain ID successfully	passed
5	Add Images	Add images details	successfully	passed

Unit Testing and other Classes of Testing

Black box Testing:

Black box testing is done to find:

- Incorrect or missing functions.
- Interface Errors
- Errors in external database access
- Performance error.
- Initialization and termination error.

White box Testing:

White box testing is done to find out:

- Check whether all independent paths within a module have been exercised at least once
- Exercise all logical decision on their true and false sides
- Execute all loops at their boundaries and within their bounds.
- Execute all loops at their boundaries and within their bounds.
- Ensure whether all the possible validity checks and validity lookups have been providedto validate data entry.

Unit Testing:

Unit testing involves, checking all the modules in the system individually against the specification produced during the design of the module and for their performance. Unit testing also involves code produced in the coding phase and hence the internal logic of the program. Each module is tested for different test cases design to check each specific combination of conditions handled by the program. Error handlers are included in each module for each event trap and handled the errors.

Integration Testing

Integration testing takes as its input modules that have been checked out by unit testing, groups them in larger aggregates, applies tests defined in an Integration test plan to those aggregates, and delivers as its output the integrated system ready for system testing. The purpose of integration testing is to verify functional, performance and reliability requirements placed on major design items.

System Testing

System testing of software or hardware is testing conducted on a complete, integrated system to evaluate the system's compliance with its specified requirements. System testing fallswithin the scope of black box testing, and as such, should require no knowledge of the inner design of the code or logic. As a rule, system testing takes, as its input, all of the "integrated" software components that have successfully passed integration testing and also the software system itself integrated with any applicable hardware system(s). The purpose of integration testing is to detect any inconsistencies between the software units that are integrated together (called *assemblages*) or between any of the assemblages and the hardware. System testing is a more limited type of testing; it seeks to detect defects both within the "inter-assemblages" and also within the system as a whole.

CHAPTER7:
SYSTEM SECURITY

CHECKS AND CONTROL:

The protection of computer based resources that includes hardware, software, data, procedures and people against unauthorized use or natural Disaster is known as System Security.

System Security can be divided into four related issues:

- Security
- Integrity
- Privacy
- Confidentiality

SYSTEM SECURITY:

It refers to the technical innovations and procedures applied to the hardware andoperation systems to protect against deliberate or accidental damage from a defined threat.

DATA SECURITY:

It is the protection of data from loss, disclosure, modification and destruction.

SYSTEM INTEGRITY:

It refers to programs, appropriate Physical security and safety against external threats such as eaves dropping and wiretapping.

PRIVACY:

It's important to note that user privacy is a top priority. Data is anonymized and aggregated to ensure individual user identities are not disclosed. It defines the rights of the user or organizations to determine what information they are willing to share with or accept from others and how the organization can be protected against unwelcome,unfair orexcessive

dissemination of information about it.

CONFIDENTIALITY:

It is a special status given to sensitive information in a database to minimize the possible invasion of privacy. It is an attribute of information that characterizes its need for protection.

ENCRYPTION SECURE:

In your web application we have provided security such as session this Session is used all web forms.

Session can be used to provide page level security means we will check In every page that the session value present or not session id in the sense Admin _id or manager _id if the admin _idormanager _id is null then redirectto login page.

CHAPTER 8: CONCLUSION

LIMITATIONS:

Hardware/software: This includes cost of sophisticated computer etc.

Maintenance: This includes cost involving maintainance.

FUTURE SCOPE:

The scope of digital identity using blockchain technology encompasses a wide range of features and capabilities that leverage the advantages of blockchain for secure, decentralized, and user-controlled identity management Biometric Integration**.**

- User Registration and Onboarding:
  1. User enrollment and identity creation on the blockchain.
  2. Verification of user identity through trusted sources or agencies.
- Blockchain-Based Identity Records:
  1. Storing user identity data on the blockchain in a secure and tamper-proof manner.
  2. Linking cryptographic keys to user identities for authentication.
- Smart Contracts for Identity Verification
- Cross-Industry Applicability:
    1. Designing the system to serve various industries and use cases, such as finance, healthcare, government services, and more.

BIBLIOGRAPHY:

Websites:

[1] https://www.javatpoint.com/python-tutorial

[2] https://www.postgresql.org

[3] https://www.w3schools.com/html

[4] https://www.geeksforgeeks.org/blockchain/